



Pingopro Partner App

Complete Development Guide

[Database](#) · [REST API](#) · [Live Map](#) · [GPS Tracking](#) · [Notifications](#) · [Production](#)

Version

Platform

Backend

Maps

Push

© 2026 Pingopro. All rights reserved.

Table of Contents

Chapter 1 — Understanding the Architecture

- What is a database?
- What is an API?
- How Android connects

Chapter 2 — Install Tools

- Node.js
- PostgreSQL
- VS Code

Chapter 3 — Database Setup

- Create database
- Create tables
- Insert test data

Chapter 4 — Build the REST API

- Project structure
- Database pool
- Auth routes
- Orders, Earnings, Vehicle routes

Chapter 5 — Connect Android to API

- Update base URL
- Allow HTTP
- Fix LoginActivity

Chapter 6 — Google Maps & Live Map Screen

- Get API key
- Map layout
- LiveMapFragment
- Draw routes

Chapter 7 — GPS Live Tracking

- Foreground service
- LocationTrackingService
- Start/stop from HomeFragment

Chapter 8 — WebSocket — Real-Time Location

- Backend WebSocket server
- LocationSender in Android

- Auto-reconnect

Chapter 9 — Firebase Push Notifications

- FCM setup
- PingoproMessagingService
- Send from backend

Chapter 10 — Remaining API Routes

- partner.js
- vehicle.js
- earnings.js

Chapter 11 — Testing the Complete App

- Test each feature
- Common errors & fixes
- Logcat debugging

Chapter 12 — Production Build & Deploy

- Signed APK
- Railway deployment
- Play Store submission

Chapter 1

Understanding the Architecture

Why you need a database, an API, and how Android connects to them

- Database (PostgreSQL)
- REST API (Node.js)
- Android App (Kotlin)
- The flow between all three

1.1 What is a database?

A database permanently saves all your app data. Partner names, order details, earnings, vehicle info — everything. Think of it as a giant Excel spreadsheet that never loses data even when you turn off the computer.

Analogy: Database = a filing cabinet in your office. Every order, every driver, every rupee earned is one file in that cabinet.

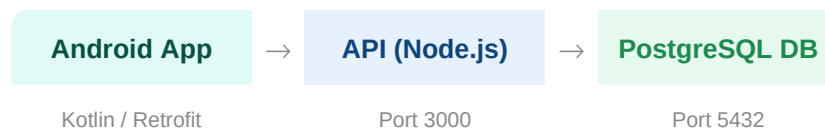
1.2 What is an API?

An API (Node.js server) is a program running on a computer 24/7. The Android app sends it requests like 'give me Rahul's orders' and the API goes to the database, gets the data, and sends it back. The app never touches the database directly.

Analogy: API = a receptionist. The customer (app) talks to the receptionist. The receptionist goes to the filing cabinet (database) and brings back what is needed.

1.3 The full data flow

Every action in the app follows this exact path:



IMPORTANT: Never connect Android directly to PostgreSQL. Always use an API in between. Putting your DB password in the app is a major security risk.

Chapter 2

Install Required Tools

One-time installation of Node.js, PostgreSQL, and VS Code

- Node.js (API language)
- PostgreSQL (database)
- VS Code (code editor)

2.1 Install Node.js

Node.js lets your computer run JavaScript as a server. This is what your API is written in.

- 1 Open your browser and go to **nodejs.org**
- 2 Download the **LTS version** (the green button — LTS = Long Term Support, the stable one)
- 3 Run the installer — keep clicking Next, leave all options as default
- 4 Open Command Prompt (Windows) or Terminal (Mac) and verify the install:

```
node --version
# You should see something like: v20.11.0

npm --version
# You should see something like: 10.2.4
```

If you see version numbers, Node.js is installed. If you see 'command not found' restart your computer and try again.

2.2 Install PostgreSQL

PostgreSQL is the database. Think of installing it like installing Microsoft Excel but for databases.

- 1 Go to [postgresql.org/download](https://www.postgresql.org/download) and choose your OS
- 2 **Windows:** Download and run the installer. Remember the password you set — write it down now.
- 3 **Mac:** Go to postgresapp.com, download Postgres.app, drag to Applications, click Initialize
- 4 After installing, open **pgAdmin** (installed with PostgreSQL) — this is your visual database dashboard

Write down your PostgreSQL password right now. If you forget it, you have to reinstall everything.

2.3 Install VS Code

- 1 Go to code.visualstudio.com and download for your OS
- 2 Install it (just click Next through everything)
- 3 Open VS Code → Extensions (4-squares icon) → install: **ESLint, Prettier, REST Client**

Chapter 3

Database Setup

Create the PostgreSQL database, all 4 tables, and insert test data

- Create pingopro_db
- partners / vehicles / orders / earnings tables
- Insert test driver and orders

3.1 Create the database in pgAdmin

- 1 Open **pgAdmin** from Start Menu / Applications
- 2 On the left: **Servers** → **PostgreSQL** → click it → enter your password
- 3 Right-click **Databases** → Create → Database
- 4 In the Name field type: **pingopro_db** → click Save
- 5 You will see **pingopro_db** appear in the left panel — database created!

3.2 Create all 4 tables

In pgAdmin click on pingopro_db → click the Query Tool button (lightning bolt icon) → paste all the SQL below → press F5:


```
-- partners table
CREATE TABLE partners (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(100),
  phone VARCHAR(15) UNIQUE NOT NULL,
  email VARCHAR(150),
  rating DECIMAL(3,2) DEFAULT 5.00,
  total_trips INTEGER DEFAULT 0,
  is_online BOOLEAN DEFAULT false,
  is_verified BOOLEAN DEFAULT false,
  wallet_balance DECIMAL(10,2) DEFAULT 0.00,
  otp_code VARCHAR(6),
  otp_expires TIMESTAMP,
  fcm_token TEXT,
  last_lat DECIMAL(10,8),
  last_lng DECIMAL(11,8),
  last_seen TIMESTAMP,
  created_at TIMESTAMP DEFAULT NOW()
);
```

```
-- vehicles table
CREATE TABLE vehicles (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  partner_id UUID REFERENCES partners(id) ON DELETE CASCADE,
  type VARCHAR(50), make VARCHAR(50),
  model VARCHAR(50), year INTEGER,
  reg_number VARCHAR(20) UNIQUE,
  capacity_kg INTEGER, is_active BOOLEAN DEFAULT true,
  rc_valid BOOLEAN DEFAULT false,
  insurance_exp DATE, permit_exp DATE,
  created_at TIMESTAMP DEFAULT NOW()
);
```

```
-- orders table
CREATE TABLE orders (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  order_id VARCHAR(20) UNIQUE NOT NULL,
  partner_id UUID REFERENCES partners(id),
  status VARCHAR(20) DEFAULT 'NEW',
  pickup_address TEXT, pickup_lat DECIMAL(10,8), pickup_lng DECIMAL(11,8),
  dropoff_address TEXT, dropoff_lat DECIMAL(10,8), dropoff_lng DECIMAL(11,8),
  estimated_fare DECIMAL(10,2), actual_fare DECIMAL(10,2),
  distance_km DECIMAL(8,2), vehicle_type VARCHAR(50),
  customer_name VARCHAR(100), customer_phone VARCHAR(15),
  scheduled_time TIMESTAMP,
  created_at TIMESTAMP DEFAULT NOW(), completed_at TIMESTAMP
);

-- earnings table
CREATE TABLE earnings (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  partner_id UUID REFERENCES partners(id),
  order_id UUID REFERENCES orders(id),
  amount DECIMAL(10,2), bonus_type VARCHAR(50),
  earned_at TIMESTAMP DEFAULT NOW()
);
```

After pressing F5 you should see 'Query returned successfully'. All 4 tables are now created!

3.3 Insert test data

```
-- Test partner (driver)
INSERT INTO partners (name, phone, is_verified, wallet_balance, total_trips, rating)
VALUES ('Rahul Sharma', '9876543210', true, 1240.00, 47, 4.8);

-- Test vehicle for Rahul
INSERT INTO vehicles (partner_id, type, make, model, year, reg_number, capacity_kg, rc_valid)
SELECT id, 'Mini Truck', 'Tata', 'Ace', 2022, 'MH31AB1234', 750, true
FROM partners WHERE phone = '9876543210';

-- Test orders
INSERT INTO orders
(order_id, status, pickup_address, dropoff_address,
estimated_fare, distance_km, vehicle_type, customer_name, customer_phone)
VALUES
('PPO-5891','IN_TRANSIT','Dharampeth, Nagpur','Sitabuldi, Nagpur',
480.00, 6.2, 'Mini Truck', 'Suresh Mehta', '9111222333'),
('PPO-5892','NEW','Itwari, Nagpur','Civil Lines, Nagpur',
360.00, 4.1, 'Mini Truck', 'Anita Rao', '9444555666');
```

Chapter 4

Build the REST API

Create a Node.js + Express backend with all routes the Android app needs

- Project structure
- DB pool
- Auth (OTP + JWT)
- Orders, Partner, Earnings, Vehicle routes
- Start the server

4.1 Create backend project

```
# Open Command Prompt / Terminal
cd Desktop
mkdir pingopro-backend
cd pingopro-backend
npm init -y

# Install all required packages
npm install express pg dotenv jsonwebtoken cors helmet ws
npm install --save-dev nodemon

# Open in VS Code
code .
```

4.2 Project folder structure

```
pingopro-backend/  
  ■■■ src/  
  ■ ■■■ routes/  
  ■ ■ ■■■ auth.js # OTP login  
  ■ ■ ■■■ partner.js # profile, online status  
  ■ ■ ■■■ orders.js # active, accept, deliver  
  ■ ■ ■■■ earnings.js # summary, payout  
  ■ ■ ■■■ vehicle.js # vehicle info  
  ■ ■■■ middleware/  
  ■ ■ ■■■ auth.js # JWT token check  
  ■ ■■■ db/  
  ■ ■ ■■■ pool.js # database connection  
  ■ ■■■ websocket.js # real-time location  
  ■ ■■■ firebase.js # push notifications  
  ■ ■■■ app.js # main entry point  
  ■■■ .env # passwords (never share!)  
  ■■■ .gitignore  
  ■■■ package.json
```

4.3 .env file — your secrets

```
# .env — fill in YOUR actual values  
DB_HOST=localhost  
DB_PORT=5432  
DB_NAME=pingopro_db  
DB_USER=postgres  
DB_PASSWORD=your_postgresql_password_here  
JWT_SECRET=make_up_a_long_random_string_abc123xyz  
PORT=3000
```

Create a .gitignore file and add 'node_modules/' and '.env' so your passwords are never uploaded to GitHub.

4.4 src/db/pool.js — database connection

```
// src/db/pool.js
const { Pool } = require('pg');
require('dotenv').config();

const pool = new Pool({
  host: process.env.DB_HOST,
  port: process.env.DB_PORT,
  database: process.env.DB_NAME,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
});

pool.connect((err, client, release) => {
  if (err) console.error('DB FAILED:', err.message);
  else { console.log('Connected to PostgreSQL!'); release(); }
});

module.exports = pool;
```

4.5 src/middleware/auth.js — JWT protection

```
// src/middleware/auth.js
const jwt = require('jsonwebtoken');

module.exports = (req, res, next) => {
  const authHeader = req.headers['authorization'];
  if (!authHeader || !authHeader.startsWith('Bearer '))
    return res.status(401).json({ success: false, message: 'No token' });

  const token = authHeader.split(' ')[1];
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.partnerId = decoded.partnerId;
    next();
  } catch (err) {
    return res.status(401).json({ success: false, message: 'Invalid token' });
  }
};
```

4.6 src/routes/auth.js — OTP login

```
const express = require('express');
const router = express.Router();
const pool = require('../db/pool');
const jwt = require('jsonwebtoken');
require('dotenv').config();

// POST /v1/auth/send-otp
router.post('/send-otp', async (req, res) => {
  const { phone } = req.body;
  const otp = Math.floor(100000 + Math.random() * 900000).toString();
  const expires = new Date(Date.now() + 5 * 60 * 1000);
  await pool.query(
    `INSERT INTO partners (phone, otp_code, otp_expires)
    VALUES ($1,$2,$3)
    ON CONFLICT (phone) DO UPDATE
    SET otp_code=$2, otp_expires=$3`,
    [phone, otp, expires]
  );
  console.log(`OTP for ${phone}: ${otp}`); // dev only
  res.json({ success: true, message: 'OTP sent' });
});

// POST /v1/auth/verify-otp
router.post('/verify-otp', async (req, res) => {
  const { phone, otp } = req.body;
  const { rows } = await pool.query(
    'SELECT * FROM partners WHERE phone=$1', [phone]
  );
  const partner = rows[0];
  if (!partner || partner.otp_code !== otp)
    return res.status(401).json({ success: false, message: 'Wrong OTP' });
  if (new Date() > new Date(partner.otp_expires))
    return res.status(401).json({ success: false, message: 'OTP expired' });

  const token = jwt.sign(
    { partnerId: partner.id, phone: partner.phone },
    process.env.JWT_SECRET, { expiresIn: '30d' }
  );
  res.json({ success: true, data: { token, partnerId: partner.id, partner } });
});

module.exports = router;
```

4.7 src/app.js — main entry point

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const helmet = require('helmet');
const http = require('http');
const { initWebSocket } = require('./websocket');
const app = express();

app.use(cors()); app.use(helmet()); app.use(express.json());

app.get('/', (req, res) => res.json({ message: 'Pingopro API running!' }));

app.use('/v1/auth', require('./routes/auth'));
app.use('/v1/partner', require('./routes/partner'));
app.use('/v1/orders', require('./routes/orders'));
app.use('/v1/earnings', require('./routes/earnings'));
app.use('/v1/vehicle', require('./routes/vehicle'));

const server = http.createServer(app);
initWebSocket(server);
server.listen(process.env.PORT || 3000, () =>
  console.log('Pingopro running on port 3000'));
```

Update package.json scripts:

```
"scripts": {
  "start": "node src/app.js",
  "dev": "nodemon src/app.js"
}

# Then start your server:
npm run dev

# You should see both lines:
# Pingopro running on port 3000
# Connected to PostgreSQL!
```

Both lines MUST appear. If only the first appears, check your .env DB_PASSWORD value.

Chapter 5

Connect Android App to Your API

Update the base URL, allow HTTP for testing, and wire the login flow

- Update Repository.kt
- network_security_config.xml
- LoginActivity real API calls

5.1 Find the correct URL for your emulator or phone

Emulator (AVD)

http://10.0.2.2:3000/v1/

Real phone (same WiFi)

http://192.168.x.x:3000/v1/

Production (Railway)

https://your-app.up.railway.app/v1/

Find local IP (Windows)

ipconfig → IPv4 Address

5.2 Update Repository.kt

Open: app → src → main → java → com.pingopro.partner → data → repository → Repository.kt

```
// Find this line and change it:
private const val BASE_URL = "https://api.pingopro.com/v1/"

// Change to (for emulator testing):
private const val BASE_URL = "http://10.0.2.2:3000/v1/"

// For real phone on same WiFi (use YOUR local IP):
private const val BASE_URL = "http://192.168.1.105:3000/v1/"
```


5.3 Allow HTTP connections (for local testing)

Open res → xml → network_security_config.xml and replace all content:

```
<?xml version="1.0" encoding="utf-8"?>
<network-security-config>
<base-config cleartextTrafficPermitted="true">
<trust-anchors>
<certificates src="system" />
<certificates src="user" />
</trust-anchors>
</base-config>
</network-security-config>
```

This is for testing only. When you deploy with HTTPS, revert this file back to the original.

5.4 Update LoginActivity.kt with real API calls

```
// Replace sendOtp() function:
private val repo = PingoproRepository()

private fun sendOtp(phone: String) {
    phoneNumber = phone
    binding.progressBar.visibility = View.VISIBLE
    lifecycleScope.launch {
        try {
            val response = repo.sendOtp(phone)
            binding.progressBar.visibility = View.GONE
            if (response.isSuccessful) {
                isOtpSent = true
                binding.otpLayout.visibility = View.VISIBLE
                binding.btnSendOtp.text = "Verify OTP"
            }
        } catch (e: Exception) {
            binding.progressBar.visibility = View.GONE
            Toast.makeText(this@LoginActivity,
                "Cannot connect: ${e.message}", Toast.LENGTH_LONG).show()
        }
    }
}

// Replace verifyOtp() function:
private fun verifyOtp(otp: String) {
    lifecycleScope.launch {
        val response = repo.verifyOtp(phoneNumber, otp)
        if (response.isSuccessful && response.body()?.success == true) {
            val data = response.body()!!.data!!
            session.saveSession(data.token, data.partnerId, data.partner.name, phoneNumber)
            startActivity(Intent(this@LoginActivity, MainActivity::class.java))
            finish()
        }
    }
}
```

Chapter 6

Google Maps & Live Map Screen

Get your API key, build the map layout, and show the driver's live location

- Get API key from Google Cloud
- Enable Maps SDK + Directions API
- fragment_live_map.xml layout
- LiveMapFragment.kt complete code

6.1 Get Google Maps API key

- 1 Go to **console.cloud.google.com** and sign in with Google account
- 2 Click **Select a project** → **New Project** → name it **Pingopro** → Create
- 3 Go to **APIs and Services** → **Library**. Enable: **Maps SDK for Android**, **Directions API**, **Geocoding API**
- 4 Go to **APIs and Services** → **Credentials** → **Create Credentials** → **API Key**. Copy the key.
- 5 Click **Restrict Key** → **Android apps** → add package name: **com.pingopro.partner**

You get \$200 free credit per month. You need to add a billing card to activate but you will not be charged unless you exceed this.

6.2 Add API key to AndroidManifest.xml

```
<!-- AndroidManifest.xml – find this line and replace the value -->
<meta-data
  android:name="com.google.android.geo.API_KEY"
  android:value="AIzaSyYOUR_ACTUAL_KEY_HERE" />

<!-- Also verify in app/build.gradle dependencies: -->
implementation 'com.google.android.gms:play-services-maps:18.2.0'
implementation 'com.google.android.gms:play-services-location:21.0.1'
```

6.3 LiveMapFragment.kt — part 1: setup & map ready

```
// ui/map/LiveMapFragment.kt
class LiveMapFragment : Fragment(), OnMapReadyCallback {

  private var map: GoogleMap? = null
  private var currentMarker: Marker? = null
  private lateinit var fusedClient: FusedLocationProviderClient
  private lateinit var locationCallback: LocationCallback

  override fun onCreateView(view: View, s: Bundle?) {
    super.onCreateView(view, s)
    fusedClient = LocationServices
      .getFusedLocationProviderClient(requireActivity())
    val mapFrag = childFragmentManager
      .findFragmentById(R.id.mapFragment) as SupportMapFragment
    mapFrag.getMapAsync(this)
    setupLocationCallback()
    checkPermissionsAndStart()
  }

  override fun onMapReady(googleMap: GoogleMap) {
    map = googleMap
    map?.uiSettings?.isZoomControlsEnabled = true
    map?.uiSettings?.isCompassEnabled = true
  }
```

```
// LiveMapFragment.kt – part 2: location updates
private fun setupLocationCallback() {
    locationCallback = object : LocationCallback() {
        override fun onLocationResult(result: LocationResult) {
            result.lastLocation?.let { loc ->
                val latLng = LatLng(loc.latitude, loc.longitude)
                updateMapPosition(latLng)
                reverseGeocode(loc.latitude, loc.longitude)
                LocationSender.send(loc.latitude, loc.longitude)
            }
        }
    }
}

private fun updateMapPosition(latLng: LatLng) {
    currentMarker?.remove()
    currentMarker = map?.addMarker(
        MarkerOptions().position(latLng).title("You are here")
            .icon(BitmapDescriptorFactory.defaultMarker(
                BitmapDescriptorFactory.HUE_CYAN)))
    map?.animateCamera(CameraUpdateFactory.newLatLngZoom(latLng, 16f))
}

private fun reverseGeocode(lat: Double, lng: Double) {
    val addresses = Geocoder(requireContext(), Locale.getDefault())
        .getFromLocation(lat, lng, 1)
    if (!addresses.isNullOrEmpty())
        binding.tvCurrentAddress.text = addresses[0].getAddressLine(0)
}

private fun startLocationUpdates() {
    val req = LocationRequest.Builder(
        Priority.PRIORITY_HIGH_ACCURACY, 5000L
    ).setMinUpdateIntervalMillis(3000L).build()
    fusedClient.requestLocationUpdates(req, locationCallback, null)
}
}
```

Chapter 7

GPS Live Tracking (Foreground Service)

Keep location tracking running even when the app is in background or locked

- What is a Foreground Service
- LocationTrackingService.kt
- Start/stop from HomeFragment

7.1 What is a Foreground Service?

When the driver closes the app or locks the phone, Android stops the app to save battery. A Foreground Service keeps running in the background and shows a persistent notification ('Pingopro is tracking your location') so Android does not kill it. This is exactly how Uber, Porter, and all delivery apps do live tracking.

7.2 LocationTrackingService.kt — part 1: setup

```
// utils/LocationTrackingService.kt
class LocationTrackingService : Service() {
    private lateinit var fusedClient: FusedLocationProviderClient
    private lateinit var locationCallback: LocationCallback
    private val scope = CoroutineScope(Dispatchers.IO + SupervisorJob())

    companion object {
        const val ACTION_START = "ACTION_START"
        const val ACTION_STOP = "ACTION_STOP"
        const val NOTIF_ID = 2001
    }

    override fun onCreate() {
        super.onCreate()
        fusedClient = LocationServices.getFusedLocationProviderClient(this)
        locationCallback = object : LocationCallback() {
            override fun onLocationResult(result: LocationResult) {
                result.lastLocation?.let { loc ->
                    scope.launch {
                        LocationSender.send(loc.latitude, loc.longitude)
                    }
                }
            }
        }
    }
}
```

```
// LocationTrackingService.kt – part 2: start/stop/notify
override fun onStartCommand(intent: Intent?, flags: Int, id: Int): Int {
    when (intent?.action) {
        ACTION_START -> {
            startForeground(NOTIF_ID, buildNotification())
            startUpdates()
        }
        ACTION_STOP -> { stopForeground(true); stopSelf() }
    }
    return START_STICKY
}

private fun startUpdates() {
    val req = LocationRequest.Builder(
        Priority.PRIORITY_HIGH_ACCURACY, 10_000L
    ).setMinUpdateIntervalMillis(5_000L).build()
    fusedClient.requestLocationUpdates(
        req, locationCallback, Looper.getMainLooper()
    )
}

private fun buildNotification() =
    NotificationCompat.Builder(this, PingoproApplication.CHANNEL_LOCATION)
        .setContentTitle("Pingopro Partner – Active")
        .setContentText("Tracking your location for trip delivery")
        .setSmallIcon(R.drawable.ic_vehicle)
        .setOngoing(true).build()

override fun onBind(i: Intent?) = null
override fun onDestroy() {
    fusedClient.removeLocationUpdates(locationCallback)
    scope.cancel()
}
}
```

7.3 Start/stop service from HomeFragment


```
// Add to HomeFragment.kt
private fun startTracking() {
    Intent(requireContext(), LocationTrackingService::class.java).also {
        it.action = LocationTrackingService.ACTION_START
        requireContext().startForegroundService(it)
    }
}

private fun stopTracking() {
    Intent(requireContext(), LocationTrackingService::class.java).also {
        it.action = LocationTrackingService.ACTION_STOP
        requireContext().startService(it)
    }
}

// Inside setupClickListeners() – wire to online toggle:
binding.switchOnline.setOnCheckedChangeListener { _, isChecked ->
    viewModel.toggleOnlineStatus(session.getToken(), isChecked)
    if (isChecked) startTracking() else stopTracking()
}
```

Chapter 8

WebSocket — Real-Time Location

Keep a permanent connection between app and server for instant location updates and trip requests

- Install 'ws' in backend
- `src/websocket.js` server
- `LocationSender.kt` Android client
- Auto-reconnect

8.1 Install WebSocket in backend

```
npm install ws
```

8.2 `src/websocket.js` — backend server

```

const { WebSocketServer } = require('ws');
const jwt = require('jsonwebtoken');
const pool = require('./db/pool');

const clients = new Map(); // partnerId -> WebSocket

function initWebSocket(server) {
  const wss = new WebSocketServer({ server });
  wss.on('connection', (ws) => {
    let partnerId = null;
    ws.on('message', async (raw) => {
      const msg = JSON.parse(raw);
      if (msg.type === 'AUTH') {
        const decoded = jwt.verify(msg.token, process.env.JWT_SECRET);
        partnerId = decoded.partnerId;
        clients.set(partnerId, ws);
        ws.send(JSON.stringify({ type: 'AUTH_OK' }));
      }
      if (msg.type === 'LOCATION' && partnerId) {
        await pool.query(
          'UPDATE partners SET last_lat=$1, last_lng=$2, last_seen=NOW() WHERE id=$3',
          [msg.lat, msg.lng, partnerId]
        );
      }
    });
    ws.on('close', () => { if (partnerId) clients.delete(partnerId); });
  });

  function sendTripRequest(partnerId, order) {
    const ws = clients.get(partnerId);
    if (ws?.readyState === 1)
      ws.send(JSON.stringify({ type: 'TRIP_REQUEST', order }));
  }

  module.exports = { initWebSocket, sendTripRequest };
}

```

8.3 Add columns to partners table

```

-- Run in pgAdmin Query Tool:
ALTER TABLE partners
ADD COLUMN last_lat DECIMAL(10,8),
ADD COLUMN last_lng DECIMAL(11,8),
ADD COLUMN last_seen TIMESTAMP;

```

8.4 utils/LocationSender.kt — Android WebSocket client

```
// utils/LocationSender.kt
object LocationSender {
    private val client = OkHttpClient()
    private var websocket: WebSocket? = null
    private var isAuthenticated = false

    fun connect(token: String, serverUrl: String) {
        val wsUrl = serverUrl
            .replace("http://", "ws://")
            .replace("https://", "wss://")
            .replace("/v1/", "")
        val request = Request.Builder().url(wsUrl).build()
        websocket = client.newWebSocket(request, object : WebSocketListener() {
            override fun onOpen(ws: WebSocket, response: Response) {
                ws.send(JSONObject().put("type", "AUTH").put("token", token).toString())
            }
            override fun onMessage(ws: WebSocket, text: String) {
                val msg = JsonObject(text)
                when (msg.getString("type")) {
                    "AUTH_OK" -> isAuthenticated = true
                    "TRIP_REQUEST" -> handleTripRequest(msg)
                }
            }
            override fun onFailure(ws: WebSocket, t: Throwable, r: Response?) {
                android.os.Handler(android.os.Looper.getMainLooper())
                    .postDelayed({ connect(token, serverUrl) }, 5000)
            }
        })
    }

    fun send(lat: Double, lng: Double) {
        if (!isAuthenticated) return
        websocket?.send(JSONObject().put("type", "LOCATION")
            .put("lat", lat).put("lng", lng).toString())
    }

    fun disconnect() { websocket?.close(1000, "Offline"); isAuthenticated = false }
}
```

Chapter 9

Firebase Push Notifications

Send trip request alerts to driver even when app is closed using FCM

- Create Firebase project
- Add google-services.json
- PingoproMessagingService.kt
- Send from Node.js backend

9.1 Set up Firebase project

- 1 Go to **console.firebase.google.com** → click **Add Project** → name it **Pingopro**
- 2 Inside the project click **Add App** → **Android**. Package name: **com.pingopro.partner**
- 3 Download **google-services.json** and copy it into your project's **app/** folder
- 4 Add to root build.gradle (inside buildscript → dependencies): **classpath 'com.google.gms:google-services:4.4.0'**
- 5 Add to top of app/build.gradle (line 2): **apply plugin: 'com.google.gms.google-services'**
- 6 Add to app/build.gradle dependencies: **implementation platform('com.google.firebase:firebase-bom:32.7.0')** and **implementation 'com.google.firebase:firebase-messaging-ktx'**
- 7 Click **Sync Now** in Android Studio

9.2 PingoproMessagingService.kt

```
// utils/PingoproMessagingService.kt
class PingoproMessagingService : FirebaseMessagingService() {

    override fun onNewToken(token: String) {
        super.onNewToken(token)
        val session = SessionManager(this)
        if (session.isLoggedIn()) {
            CoroutineScope(Dispatchers.IO).launch {
                PingoproRepository().saveFcmToken(session.getToken(), token)
            }
        }
    }

    override fun onMessageReceived(message: RemoteMessage) {
        val data = message.data
        when (data["type"]) {
            "TRIP_REQUEST" -> NotificationHelper.showTripRequestNotification(
                applicationContext,
                data["orderId"] ?: "",
                data["pickup"] ?: "",
                data["dropoff"] ?: "",
                data["fare"] ?: ""
            )
        }
    }
}
```

Add this in AndroidManifest.xml inside the tag:

```
<service android:name=".utils.PingoproMessagingService" android:exported="false">
    <intent-filter>
        <action android:name="com.google.firebase.MESSAGING_EVENT" />
    </intent-filter>
</service>
```

9.3 Send FCM from Node.js backend

```
# Install in backend:
npm install firebase-admin

# Download service account key from Firebase Console →
# Project Settings → Service Accounts → Generate new private key
# Save as firebase-key.json in your backend root folder
```

```
// src/firebase.js
const admin = require('firebase-admin');
const serviceAccount = require('../firebase-key.json');
admin.initializeApp({ credential: admin.credential.cert(serviceAccount) });

async function sendTripNotification(fcmToken, orderId, pickup, dropoff, fare) {
  await admin.messaging().send({
    token: fcmToken,
    data: { type: 'TRIP_REQUEST', orderId, pickup, dropoff, fare: fare.toString() },
    notification: { title: 'New trip request!', body: `${pickup} to ${dropoff}` },
    android: { priority: 'high', notification: { channelId: 'pingopro_trip_requests' } }
  });
}
module.exports = { sendTripNotification };
```

Chapter 10

Remaining API Routes

Complete backend routes for partner profile, vehicle, orders, and earnings

- `src/routes/partner.js`
- `src/routes/vehicle.js`
- `src/routes/orders.js`
- `src/routes/earnings.js`

10.1 `src/routes/partner.js`


```
const express = require('express'), router = express.Router();
const pool = require('../db/pool'), auth = require('../middleware/auth');

// GET /v1/partner/profile
router.get('/profile', auth, async (req, res) => {
  const { rows } = await pool.query(
    `SELECT p.*, v.type, v.make, v.model, v.reg_number, v.capacity_kg
    FROM partners p LEFT JOIN vehicles v
    ON v.partner_id=p.id AND v.is_active=true WHERE p.id=$1`,
    [req.partnerId]
  );
  const p = rows[0];
  res.json({ success: true, data: {
    id: p.id, name: p.name, phone: p.phone,
    rating: p.rating, totalTrips: p.total_trips,
    isOnline: p.is_online, isVerified: p.is_verified,
    joinedDate: p.created_at,
    wallet: { balance: p.wallet_balance }
  }});
});

// PUT /v1/partner/status
router.put('/status', auth, async (req, res) => {
  await pool.query(
    'UPDATE partners SET is_online=$1 WHERE id=$2',
    [req.body.isOnline, req.partnerId]
  );
  res.json({ success: true });
});

// POST /v1/partner/fcm-token
router.post('/fcm-token', auth, async (req, res) => {
  await pool.query('UPDATE partners SET fcm_token=$1 WHERE id=$2',
    [req.body.token, req.partnerId]);
  res.json({ success: true });
});

module.exports = router;
```

10.2 src/routes/orders.js

```
// GET /v1/orders/active
router.get('/active', auth, async (req, res) => {
  const { rows } = await pool.query(
    `SELECT * FROM orders WHERE partner_id=$1
    AND status IN ('ACCEPTED','IN_TRANSIT') ORDER BY created_at DESC`,
    [req.partnerId]
  );
  res.json({ success: true, data: rows });
});

// POST /v1/orders/:id/accept
router.post('/:id/accept', auth, async (req, res) => {
  const { rows } = await pool.query(
    `UPDATE orders SET status='ACCEPTED', partner_id=$1
    WHERE id=$2 AND status='NEW' RETURNING *`,
    [req.partnerId, req.params.id]
  );
  res.json({ success: true, data: rows[0] });
});

// POST /v1/orders/:id/deliver
router.post('/:id/deliver', auth, async (req, res) => {
  const { rows } = await pool.query(
    `UPDATE orders SET status='DELIVERED', completed_at=NOW()
    WHERE id=$1 AND partner_id=$2 RETURNING *`,
    [req.params.id, req.partnerId]
  );
  await pool.query(
    'INSERT INTO earnings (partner_id, order_id, amount) VALUES ($1,$2,$3)',
    [req.partnerId, req.params.id, rows[0].actual_fare || rows[0].estimated_fare]
  );
  res.json({ success: true, data: rows[0] });
});
```

10.3 src/routes/earnings.js

```
// GET /v1/earnings/summary
router.get('/summary', auth, async (req, res) => {
  const pid = req.partnerId;
  const today = await pool.query(
    `SELECT COALESCE(SUM(amount),0) AS total FROM earnings
    WHERE partner_id=$1 AND earned_at::date=CURRENT_DATE`, [pid]);
  const week = await pool.query(
    `SELECT COALESCE(SUM(amount),0) AS total FROM earnings
    WHERE partner_id=$1 AND earned_at>=NOW()-INTERVAL '7 days'`, [pid]);
  const partner = await pool.query(
    'SELECT wallet_balance FROM partners WHERE id=$1', [pid]);
  res.json({ success: true, data: {
    todayEarnings: parseFloat(today.rows[0].total),
    weeklyEarnings: parseFloat(week.rows[0].total),
    walletBalance: parseFloat(partner.rows[0].wallet_balance),
    peakHourBonus: 120, weeklyTargetBonus: 250, referralCredit: 100
  }});
});

// POST /v1/earnings/payout
router.post('/payout', auth, async (req, res) => {
  await pool.query('UPDATE partners SET wallet_balance=0 WHERE id=$1',[req.partnerId]);
  res.json({ success: true, message: 'Payout requested. Funds in 24 hours.' });
});
```

Chapter 11

Testing the Complete App

Verify every feature works from API to Android end-to-end

- Test API with curl
- Run on emulator
- Logcat debugging
- Common errors and fixes

11.1 Test API endpoints first (before Android)

```
# 1. Start your backend
cd pingopro-backend && npm run dev

# 2. Open browser → http://localhost:3000
# Should show: { "message": "Pingopro API running!" }

# 3. Test send-OTP (Windows cmd):
curl -X POST http://localhost:3000/v1/auth/send-otp ^
-H "Content-Type: application/json" ^
-d '{"phone":"9876543210"}'
# Expected: { "success": true, "message": "OTP sent" }
# Check your API terminal – the OTP prints there!

# 4. Verify OTP (use the number from terminal):
curl -X POST http://localhost:3000/v1/auth/verify-otp ^
-H "Content-Type: application/json" ^
-d '{"phone":"9876543210","otp":"384726"}'
# Expected: { "success": true, "data": { "token": "eyJ..." } }
```

11.2 Common errors and fixes

Error: CLEARTEXT communication not permitted	Fix: Open network_security_config.xml and add cleartextTrafficPermitted="true"
Error: Connection refused / Failed to connect	Fix: Backend is not running. Run: npm run dev in VS Code terminal
Error: Unable to resolve host 10.0.2.2	Fix: Emulator has no internet. Wipe emulator data (AVD Manager → Wipe Data) and restart
Error: 401 Unauthorized	Fix: JWT token is wrong or expired. Log out and log in again in the app
Error: 404 Not Found	Fix: Wrong URL path. Check BASE_URL ends with /v1/ and route names match
Error: GPS dot not moving on map	Fix: Check location permission is GRANTED. Check startLocationUpdates() is called after permission.

11.3 Complete feature checklist

- Login with phone number — OTP appears in API terminal
- Enter OTP in app — reaches Home screen
- Home screen shows partner name and rating from database
- Online/Offline toggle updates is_online in partners table
- Live Map screen opens and shows blue dot at your location
- Address reverse-geocodes and appears below the map
- GPS service shows persistent notification when online
- Location updates appear in last_lat/last_lng in DB every 5–10 sec
- Orders screen shows active orders from database
- Accept button updates order status to ACCEPTED
- Mark Delivered creates an earnings row in DB
- Earnings screen loads summary with correct totals
- Vehicle screen shows your vehicle details and document status
- Profile screen shows correct name, rating, and logout works

-
- Push notification appears when a trip is assigned

Chapter 12

Production Build & Deploy

Build a signed APK, deploy the backend live, and publish to Play Store

- Build signed release APK
- Deploy to Railway (free)
- Update Android BASE_URL
- Google Play Store submission

12.1 Build signed release APK

- 1 In Android Studio go to **Build** → **Generate Signed Bundle / APK**
- 2 Choose **APK** → Next → click **Create new** keystore → fill in the form. **Remember this password forever — you need it for every update.**
- 3 Choose **release** build variant → click Finish
- 4 The APK appears in **app/release/app-release.apk**. Share via WhatsApp/email for testing.

Before building release: update BASE_URL to your live Railway URL and revert network_security_config.xml to remove the HTTP permission.

12.2 Deploy backend to Railway (free tier)

- 1 Go to **github.com** → create a new repository called **pingopro-backend**
- 2 Push your backend code: `git init` → `git add .` → `git commit -m 'initial'` → `git push`
- 3 Go to **railway.app** → sign in with GitHub → New Project → Deploy from GitHub → select your repo

- 4 Click **+ New** → **Database** → **PostgreSQL**. Railway creates a cloud database automatically.
- 5 Click your API service → **Variables** tab → add all values from your .env file. Railway shows the DB credentials to copy.
- 6 Railway gives you a URL like: **https://pingopro-backend-production.up.railway.app**
- 7 Update **Repository.kt** **BASE_URL** to this Railway URL and rebuild the app.

12.3 Summary of all API endpoints

Method	Endpoint	Description
POST	/v1/auth/send-otp	Send OTP to phone number
POST	/v1/auth/verify-otp	Verify OTP and return JWT token
GET	/v1/partner/profile	Get partner profile (auth required)
PUT	/v1/partner/status	Update online/offline status
POST	/v1/partner/fcm-token	Save Firebase push token
GET	/v1/orders/active	Get active/in-transit orders
GET	/v1/orders/upcoming	Get scheduled orders
GET	/v1/orders/history	Get completed orders
POST	/v1/orders/:id/accept	Accept a new order
POST	/v1/orders/:id/decline	Decline a trip request
POST	/v1/orders/:id/deliver	Mark order as delivered
GET	/v1/earnings/summary	Today/week/month earnings
GET	/v1/earnings/trips	Recent trip earnings list
POST	/v1/earnings/payout	Request wallet payout
GET	/v1/vehicle	Get vehicle details and doc status

Congratulations! You now have a complete Pingopro Partner app with database, REST API, live GPS map, real-time WebSocket tracking, push notifications, and a production deployment. The app is ready for real drivers!

For further help with Razorpay payouts, Twilio SMS, Google Play Store publishing, or adding a customer-facing app, continue the conversation in Claude.